

Warehouse and SQL

**Common Patterns.
Clear Fixes.**

50 engineering patterns covering query performance, table design, statistics, concurrency, SCD patterns, surrogate keys, SQL constructs, and cross-warehouse patterns in Microsoft Fabric Warehouse. Patterns every Fabric engineer must recognise, covering the gap between how Fabric is described and how it actually behaves.

Download all books free at ssanjaychandra.com/freedownloads

INTRODUCTION

SQL behaves differently here.

Fabric Warehouse is not SQL Server. It looks like SQL Server. It accepts T-SQL syntax. It has tables, views, stored procedures, and familiar tooling. But the execution engine underneath is a distributed MPP system, and that changes almost everything about how queries perform, how data should be modeled, and what SQL constructs work the way you expect.

The most common source of engineering problems in Fabric Warehouse is engineers applying SQL Server intuitions to a system that does not share SQL Server internals. Indexes that do not exist. Statistics that behave differently. Temp tables with different scoping rules. Transactions that do not behave like OLTP transactions. T-SQL features that are silently unsupported or subtly different.

This book covers seven chapters. Chapter 1 covers query performance patterns unique to Fabric Warehouse. Chapter 2 covers table design decisions that determine whether the distributed engine works for you or against you. Chapter 3 covers statistics and execution plans, which drive most unexplained performance regressions. Chapter 4 covers concurrency and blocking, which appears when multiple workloads share the same Warehouse. Chapter 5 covers SCD patterns, surrogate key management, and MERGE behavior at scale. Chapter 6 covers SQL constructs including views, stored procedures, window functions, subqueries, and CTEs. Chapter 7 covers ingestion, cross-warehouse queries, and the SQL endpoint.

Every scenario follows the same structure: the situation, the wrong thinking, what is actually happening, and the fix. The wrong thinking section is as important as the fix because most of the wrong thinking comes from reasonable SQL Server experience being applied to the wrong system.

FABRIC ENGINEERING PATTERNS

Chapter 1

Query Performance

Seven scenarios covering why queries behave unexpectedly in Fabric Warehouse and how the distributed execution engine produces results that SQL Server intuitions do not predict.

01

QUERY PERFORMANCE

Your query ran in 2 seconds last week. This week it takes 3 minutes. Nothing changed.

THE SITUATION

A query returning aggregated sales data from a Fabric Warehouse table has been consistently fast for weeks. This week the same query with identical parameters takes nearly 3 minutes. No schema changes were made and no new data was loaded differently than before.

WRONG THINKING

Engineers assume a Fabric platform issue or capacity contention. They run the query again hoping the slowdown is transient. When it persists, a support ticket is filed before the query execution plan is examined.

WHAT IS ACTUALLY HAPPENING

Table statistics have gone stale. Fabric Warehouse uses statistics to build query execution plans. When statistics are fresh, the planner chooses an efficient join order and distribution strategy. After significant data growth or heavy DML operations, statistics no longer reflect actual data distribution. The planner makes poor decisions based on outdated row counts and skew estimates, choosing full table scans or suboptimal join strategies that were never needed before.

THE FIX

Run `UPDATE STATISTICS` on the tables involved in the slow query. Check whether auto-update statistics fired recently using system catalog views. For tables receiving heavy daily loads, schedule manual statistics updates as part of the post-load maintenance routine. After updating, compare the new execution plan against the old one to confirm the planner is now choosing the expected strategy.

02

QUERY PERFORMANCE

Your query returns correct results from the Warehouse endpoint. Different results from the SQL endpoint.

THE SITUATION

The same T-SQL query is run against a Fabric Warehouse through two different connection paths: the Warehouse endpoint and the SQL analytics endpoint. Both return results without errors but the row counts and aggregated values differ between the two. The source tables have not changed.

WRONG THINKING

Engineers assume a data sync issue between the two endpoints and check whether a recent load failed partway through. When the load history looks clean, the inconsistency feels like a Fabric bug. Both endpoints are expected to serve identical data from the same underlying storage.

WHAT IS ACTUALLY HAPPENING

The Warehouse endpoint queries committed data in the Warehouse storage layer. The SQL analytics endpoint queries the Delta tables in OneLake directly. If data was recently loaded into the Warehouse using T-SQL INSERT or COPY INTO, it is visible immediately through the Warehouse endpoint. But the SQL endpoint reads from OneLake Delta files which may not yet reflect the latest Warehouse transaction depending on the sync interval and whether the Warehouse has flushed its internal buffer to Delta format. The two endpoints are not always in sync.

THE FIX

For time-sensitive reporting that requires the latest committed data, always connect through the Warehouse endpoint rather than the SQL analytics endpoint. If the SQL endpoint must be used for reporting, validate that the data lag between Warehouse and SQL endpoint is acceptable for your use case. Add a post-load validation step that queries both endpoints and compares row counts, alerting if they diverge beyond an acceptable threshold.

03

QUERY PERFORMANCE

A CTE referenced multiple times in the same query is being evaluated multiple times.

THE SITUATION

A query uses a CTE to compute an expensive aggregation that is then referenced in two places in the outer query: once in the SELECT list and once in a WHERE clause filter. The query is significantly slower than expected given the size of the data. The CTE logic looks efficient in isolation.

WRONG THINKING

Engineers assume CTEs are cached and reused like temp tables. The CTE is defined once at the top of the query and referenced twice, which feels equivalent to computing it once and reading the result twice. The performance cost of evaluating the CTE twice is not obvious from reading the query.

WHAT IS ACTUALLY HAPPENING

In Fabric Warehouse, CTEs are not materialized. They are treated as inline view definitions and expanded into the query plan at each reference point. A CTE referenced twice in the same query is evaluated twice, performing the full computation each time. This is the same behavior as SQL Server but is surprising to engineers who assume CTEs behave like temp tables or cached subquery results. The expensive aggregation runs twice, doubling the execution cost.

THE FIX

Materialize the CTE result into a temp table or a staging table before referencing it multiple times. Write the CTE result to a temp table with `SELECT INTO` or `INSERT INTO`, then reference the temp table in the outer query. This forces the aggregation to execute once and the result to be reused. Alternatively, restructure the query to reference the CTE only once and push the filter logic inside the CTE definition.

04

QUERY PERFORMANCE

Your query returns 10 rows but the execution plan shows 50 million rows estimated.

THE SITUATION

A query that filters a large table down to 10 matching rows runs very slowly. Examining the execution plan reveals that the optimizer estimated 50 million rows would be processed at the join stage despite the filter reducing the result to 10 rows early in the plan. The actual row count and the estimated row count are orders of magnitude apart.

WRONG THINKING

Engineers look at the filter condition and confirm it is correct. The filter is there and logically it should reduce the dataset before the expensive join. The query looks like it should be fast based on reading the SQL. The plan is not examined carefully.

WHAT IS ACTUALLY HAPPENING

The query planner evaluated the filter condition as non-selective based on stale or missing statistics. Without accurate statistics on the filtered column, the optimizer does not know that the filter will reduce 500 million rows to 10. It assumes a high number of rows will pass the filter and builds a plan that allocates resources for processing millions of rows through the join. The join executes with far more memory and compute than needed, even though the actual result set is tiny. The filter runs but the plan was already sized for the worst case.

THE FIX

Update statistics on the filtered column using `UPDATE STATISTICS tablename (column_name)`. With accurate statistics, the optimizer will correctly estimate a low row count after the filter and choose a more efficient plan. If the column has highly skewed distribution, create column-level statistics explicitly rather than relying on auto-generated statistics. After updating, re-examine the plan to confirm estimated rows now closely matches actual rows.

05

QUERY PERFORMANCE

Power BI is hitting your SQL endpoint instead of the Warehouse. Reports are slow.

THE SITUATION

Power BI reports connected to a Fabric Warehouse are running slower than expected. Direct Lake mode was configured but reports still feel like they are doing full SQL queries. Checking the Power BI performance analyzer shows long query durations that look like DirectQuery behavior rather than Direct Lake.

WRONG THINKING

Engineers check the Power BI dataset connection and see it points to the correct Fabric workspace. They assume the slowness is a Power BI capacity issue and investigate report design rather than the connection path itself.

WHAT IS ACTUALLY HAPPENING

The Power BI semantic model is connecting to the SQL analytics endpoint rather than using Direct Lake mode. This can happen when the model was created by connecting to the workspace SQL endpoint string rather than by creating a semantic model directly from the Warehouse in Fabric. Direct Lake mode bypasses the SQL engine entirely and reads Delta Parquet files directly. SQL endpoint mode executes T-SQL queries against the endpoint, which adds translation and execution overhead. The two modes have dramatically different performance profiles for large datasets.

THE FIX

Verify the Power BI semantic model connection type in the dataset settings. If it shows a SQL Server connection string pointing to the SQL endpoint, the model is in DirectQuery mode against the SQL endpoint. To use Direct Lake, create the semantic model directly from the Warehouse or Lakehouse in the Fabric portal using the new semantic model option rather than connecting from Power BI Desktop with a SQL connection string. Direct Lake models read Delta files natively and avoid the SQL translation layer entirely.

06

QUERY PERFORMANCE

A subquery in your WHERE clause is causing a full table scan on every outer row.

THE SITUATION

A query filters results using a correlated subquery in the WHERE clause. On small dev data the query returns in seconds. On data at scale with millions of rows in the outer table, the query runs for hours. The subquery logic looks correct and efficient in isolation.

WRONG THINKING

Engineers test the subquery independently and it returns quickly. They assume the slowness is caused by something else in the query and optimize other parts. The subquery is not suspected because it works fast when run on its own.

WHAT IS ACTUALLY HAPPENING

A correlated subquery in a WHERE clause is evaluated once for every row in the outer query. If the outer table has 10 million rows, the subquery executes 10 million times. Even if each individual execution takes 1 millisecond, the total cost is 10,000 seconds. In Fabric Warehouse's distributed execution engine, this pattern is particularly expensive because each subquery evaluation may require data movement across distribution nodes. The subquery is fast in isolation because it runs once. In the correlated context it runs millions of times.

THE FIX

Rewrite the correlated subquery as a JOIN or a window function. Most correlated subqueries that check for existence or fetch a single aggregated value can be rewritten as an INNER JOIN, LEFT JOIN with NULL check, or a subquery in the FROM clause that is evaluated once and joined to the outer table. Use `EXISTS` with a JOIN rewrite rather than a correlated subquery wherever possible. After rewriting, compare execution plans to confirm the correlated evaluation pattern is gone.

07

QUERY PERFORMANCE

Your query uses DISTINCT. It is scanning and sorting the entire table to deduplicate.

THE SITUATION

A query uses `SELECT DISTINCT` to return unique combinations of 3 columns from a table with 100 million rows. The query takes 20 minutes to complete. The number of distinct combinations in the result is only around 5000. The result size is tiny but the execution time is enormous.

WRONG THINKING

Engineers assume `DISTINCT` is a lightweight operation because the result set is small. They focus on optimizing other parts of the query before examining how `DISTINCT` is executed against the full table.

WHAT IS ACTUALLY HAPPENING

`SELECT DISTINCT` in Fabric Warehouse requires the engine to scan the full table, redistribute data across distribution nodes by the `DISTINCT` key columns, and then sort and deduplicate within each distribution. Even if the final result has only 5000 rows, the engine must process all 100 million rows to determine which ones are unique. There is no shortcut. The full scan and redistribution cost is proportional to the source table size, not the result size. `DISTINCT` on large tables is always expensive regardless of how many distinct values exist.

THE FIX

Replace `SELECT DISTINCT` with `GROUP BY` on the same columns. `GROUP BY` performs the same deduplication but allows the query planner more optimization options including predicate pushdown and early aggregation. For frequently run deduplication queries, consider pre-computing the distinct combinations into a materialized view or a staging table that is refreshed after each load. If `DISTINCT` is genuinely needed at query time, ensure the table has statistics on the `DISTINCT` key columns so the planner can estimate redistribution cost accurately.

02

FABRIC ENGINEERING PATTERNS

Chapter 2 Table Design and Distribution

Six scenarios covering how table design decisions in Fabric Warehouse determine whether the distributed engine works for you or against you at scale.

08

TABLE DESIGN AND DISTRIBUTION

You expected hash distribution to speed up joins. It made joins slower.

THE SITUATION

Two large tables were hash-distributed on what appeared to be a logical join key. The expectation was that co-located data on matching distribution keys would eliminate data movement during joins. After implementing hash distribution, join performance did not improve and in some cases got worse.

WRONG THINKING

Engineers verify that both tables are distributed on the same column and confirm the join condition uses that column. Everything looks textbook correct. The lack of improvement is attributed to Fabric not behaving as expected rather than to a problem with the distribution choice.

WHAT IS ACTUALLY HAPPENING

The distribution key chosen has skewed data distribution. If one distribution value accounts for 40 percent of all rows, that distribution node receives 40 percent of the data while others receive a fraction. Joins on a skewed distribution key cause one node to do the majority of the work while others sit idle. The data movement that hash distribution was meant to eliminate is replaced by compute skew that is often worse. Hash distribution only helps when data is evenly distributed across the distribution values.

THE FIX

Before choosing a hash distribution key, analyze the column's value distribution using `COUNT(*) GROUP BY distribution_column ORDER BY COUNT(*) DESC`. If the top 10 values account for more than 20 to 30 percent of all rows, the column is too skewed for hash distribution. Choose a column with more even distribution, typically a high-cardinality numeric key. If no suitable column exists, round-robin distribution may produce better overall performance than a skewed hash distribution despite requiring data movement on joins.

09

TABLE DESIGN AND DISTRIBUTION

You created a non-clustered index on a Fabric Warehouse table. It was silently ignored.

THE SITUATION

A non-clustered index was created on a frequently filtered column in a Fabric Warehouse table expecting SQL Server-like index seeks to replace full table scans. After creating the index, query performance did not change. The index appears in system catalog views but queries are still doing full scans.

WRONG THINKING

Engineers verify the index exists and confirm the query filter uses the indexed column. In SQL Server this setup would produce an index seek. The lack of improvement feels like a Fabric bug or a query optimizer deficiency rather than a fundamental difference in how Fabric Warehouse handles indexes.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse does not use row-level non-clustered indexes the way SQL Server does. It is a columnar distributed system where data is stored in compressed column groups across distribution nodes. Non-clustered indexes as they exist in SQL Server row-store tables are not the performance mechanism in Fabric Warehouse. The index may be accepted syntactically but it does not change the physical access path the engine uses. Data skipping and predicate pushdown in Fabric Warehouse are driven by statistics and columnstore segment elimination, not by B-tree index seeks.

THE FIX

Remove non-clustered indexes that are not providing value. Instead, focus on statistics quality for the filtered columns using `UPDATE STATISTICS`, ensure the table is properly distributed for the join patterns used in queries, and rely on columnstore segment elimination for range-based filtering. For point lookups on specific key values that require sub-millisecond response, consider whether a Warehouse is the right serving layer or whether a KQL database or a key-value store better fits that access pattern.

10

TABLE DESIGN AND DISTRIBUTION

Your columnstore table has very few rows per row group. Compression is poor and queries are slow.

THE SITUATION

A Fabric Warehouse table was loaded incrementally with small daily batches of 10,000 rows each. After 90 days, the table has 900,000 rows but query performance is poor and the storage size is much larger than expected for a columnstore table. The data looks fragmented when checking internal metadata.

WRONG THINKING

Engineers assume the table needs more data to benefit from columnstore compression and plan to wait until volume grows. They do not connect the incremental loading pattern to the row group fragmentation problem.

WHAT IS ACTUALLY HAPPENING

Columnstore compression achieves its best compression ratios and query performance with large row groups, ideally around 1 million rows per row group. When data is loaded in small batches, each batch creates a new small delta store that is not yet compressed into a proper columnstore row group. After 90 days of 10,000-row loads, the table has 90 small uncompressed delta stores instead of a few large compressed row groups. Queries must scan all delta stores individually without the benefit of columnar compression or segment-level skipping. Small batch loads are the most common cause of poor columnstore performance.

THE FIX

Run `ALTER INDEX ALL ON tablename REBUILD` or an equivalent reorganize operation to force delta stores to be compressed into full columnstore row groups. After the rebuild, verify row group sizes using system catalog views and confirm most row groups contain close to 1 million rows. For ongoing incremental loads, batch inserts to a minimum size of 100,000 rows before loading to the Warehouse, or load to a staging table and bulk insert into the target in a single operation to ensure each load creates properly sized row groups.

11

TABLE DESIGN AND DISTRIBUTION

Your varchar columns are defined as varchar(MAX). Storage and query performance are both suffering.

THE SITUATION

A Fabric Warehouse table was created with several string columns defined as varchar(MAX) for flexibility. Over time, storage costs are higher than expected and queries filtering or grouping on these columns are noticeably slower than on numeric columns of equivalent cardinality.

WRONG THINKING

Engineers assume varchar(MAX) is a safe default that simply reserves space if needed but does not consume resources when values are short. The storage and performance impact of using MAX length on all string columns is not expected.

WHAT IS ACTUALLY HAPPENING

In Fabric Warehouse, varchar(MAX) columns affect both storage layout and query performance. Columnstore compression is less effective on MAX-length columns because the engine cannot make size assumptions during compression. Statistics on varchar(MAX) columns are less precise because histogram boundaries are harder to maintain for unbounded strings. Additionally, data movement operations during distributed joins carry more overhead when column widths are undefined. Short strings stored in varchar(MAX) columns use more storage than the same strings in appropriately sized varchar columns.

THE FIX

Audit all varchar(MAX) column definitions and replace them with the smallest practical fixed length. If a column stores country codes, define it as varchar(3). If it stores email addresses, varchar(254) is the RFC maximum. For columns where length is genuinely variable and large, varchar(MAX) may be appropriate, but these should be exceptions not defaults. Run `ALTER TABLE` to update column definitions and rebuild statistics after the change. The reduction in storage and improvement in compression can be significant for wide tables with many string columns.

12

TABLE DESIGN AND DISTRIBUTION

Your fact table and dimension table are on different distributions. Every join requires data movement.

THE SITUATION

A star schema was implemented in Fabric Warehouse with a large fact table hash-distributed on a date key and dimension tables distributed round-robin. Joins between the fact and dimension tables are slow. The execution plan shows a data movement operation on every join regardless of how the query is written.

WRONG THINKING

Engineers optimize the query by reducing selected columns and adding filters. Performance improves slightly but the data movement step persists in every execution plan. The join structure itself is not questioned because the schema design looks standard.

WHAT IS ACTUALLY HAPPENING

When two tables are joined and their data does not reside on the same distribution nodes for matching key values, Fabric Warehouse must move data across nodes to complete the join. This is called a broadcast move or a shuffle move depending on table sizes. A fact table distributed on a date key joined to a dimension table distributed round-robin means no join key aligns the data co-location. Every join execution requires the engine to either broadcast the entire dimension table to all nodes or shuffle fact table rows to match dimension rows. For large tables this data movement cost dominates total query time.

THE FIX

Co-locate frequently joined tables by distributing both on the same join key column. If the fact table is distributed on a surrogate key used in dimension joins, distribute the dimension table on the same surrogate key. For small dimension tables that are joined frequently, round-robin distribution with broadcast during joins is acceptable since broadcasting a small table is cheap. For large dimension tables, hash distribution on the join key is the correct approach. Review join patterns in your most frequent queries and design distribution to minimize data movement for those specific patterns.

13

TABLE DESIGN AND DISTRIBUTION

You partitioned your Warehouse table on a date column. The partition key has too many distinct values. Everything got worse.

THE SITUATION

A large Fabric Warehouse table was partitioned by a date column at the day level expecting partition elimination to speed up date-range queries. After partitioning, query performance for date-filtered queries did not improve and some queries got slower. The partition count grew to over 3000 as data accumulated.

WRONG THINKING

Engineers confirm partition elimination is working by checking execution plans which show partition range scans. The partitioning looks correct technically. The continued poor performance is attributed to other factors rather than the partition design itself.

WHAT IS ACTUALLY HAPPENING

Partitioning a Fabric Warehouse table at the day level for a table with 3 years of data creates over 1000 partitions. Each partition has its own metadata, its own statistics, and its own set of row groups. At query planning time, the engine must evaluate metadata for all partitions even when partition elimination reduces the scan to a small range. With thousands of partitions, metadata overhead becomes significant. Additionally, each small partition has fewer rows per row group, degrading columnstore compression effectiveness. Over-partitioning is worse than no partitioning for most Fabric Warehouse workloads.

THE FIX

Partition at month or year level rather than day level unless queries exclusively filter on single-day ranges. Monthly partitioning for 3 years of data creates 36 partitions rather than 1000. This is a manageable number that retains partition elimination benefits for month and range queries without metadata overhead. For day-level filtering, rely on columnstore segment elimination driven by statistics rather than physical partitioning. After repartitioning, rebuild statistics and verify query plans show appropriate partition pruning.

FABRIC ENGINEERING PATTERNS

Chapter 3

Statistics and Execution Plans

Seven scenarios covering how the Fabric Warehouse query planner makes decisions and why stale statistics, plan cache behavior, and estimation errors produce unexpected execution strategies.

14

STATISTICS AND EXECUTION PLANS

Auto-update statistics is on. Statistics are still stale after a large daily load.

THE SITUATION

A Fabric Warehouse table receives a large daily load that increases row count by 20 percent each day. Auto-update statistics is enabled. Despite this, queries after the load run slowly with plans that look like they were built before the new data arrived. Manually updating statistics fixes the performance until the next load.

WRONG THINKING

Engineers confirm auto-update statistics is enabled and assume it will handle updates automatically after each load. When stale statistics cause issues after every load, they assume auto-update is not working correctly and consider it a platform defect.

WHAT IS ACTUALLY HAPPENING

Auto-update statistics in Fabric Warehouse triggers based on a threshold of changed rows relative to the table size. For large tables, this threshold can be quite high in absolute terms. A table with 1 billion rows may require 200 million row changes before auto-update triggers. A daily load of 20 percent of rows on a moderately sized table may not cross the threshold every day. Additionally, auto-update statistics runs asynchronously and may not complete before the next query executes against the freshly loaded data.

THE FIX

Add an explicit `UPDATE STATISTICS tablename` step as the final activity in your daily load pipeline, after the data is committed. Do not rely on auto-update for tables with large daily loads. For very large tables where full statistics updates are slow, update statistics only on the columns used most frequently in WHERE clauses and join conditions rather than all columns. Schedule statistics updates during the maintenance window immediately after load completion rather than waiting for auto-update to trigger.

15

STATISTICS AND EXECUTION PLANS

You updated statistics. The query is still using the old execution plan.

THE SITUATION

Statistics were manually updated on a slow query's source tables. The update completed successfully. The query was run again expecting a better execution plan. The plan did not change and the query remained slow. Checking the statistics shows they are current.

WRONG THINKING

Engineers assume the statistics update did not work correctly and run it again. When the plan still does not change, the assumption shifts to a Fabric platform issue where the planner is ignoring fresh statistics.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse caches execution plans in the plan cache. After statistics are updated, existing cached plans are not automatically invalidated. The next execution of the query may reuse the cached plan that was built before the statistics update, even though newer statistics are now available. The plan remains stale until the cache entry expires or is explicitly cleared. This is not a statistics problem. It is a plan cache staleness problem.

THE FIX

After updating statistics on tables used by a slow query, clear the relevant plan from the cache or run the query with a recompile hint to force a fresh plan compilation. In Fabric Warehouse you can use `DBCC FREEPROCCACHE` to clear the entire plan cache, or run the query with `OPTION (RECOMPILE)` to force a one-time recompilation without clearing other cached plans. After confirming the new plan is correct and performs well, subsequent executions will use the new cached plan.

16

STATISTICS AND EXECUTION PLANS

The query planner chose a nested loop join on two large tables. Performance is terrible.

THE SITUATION

A join between two large Fabric Warehouse tables is running extremely slowly. Examining the execution plan reveals a nested loop join strategy. For each row in the outer table, the inner table is scanned individually. With millions of rows in the outer table, this produces millions of inner table scans.

WRONG THINKING

Engineers rewrite the query trying different join syntax and hint styles familiar from SQL Server. Some rewrites produce the same plan. Others produce errors. The nested loop join persists and the root cause remains unclear.

WHAT IS ACTUALLY HAPPENING

The nested loop join was chosen because statistics on one or both tables are severely outdated, leading the optimizer to estimate very low row counts on one side of the join. A nested loop join is efficient when one table is very small. When the optimizer incorrectly believes a large table contains only a few rows, it chooses nested loop as the strategy. At runtime the actual row count is millions, making the nested loop catastrophically expensive. The optimizer chose the right strategy for the wrong data profile.

THE FIX

Update statistics on both tables involved in the join immediately. After updating, re-examine the plan to see if the optimizer now chooses a hash join or merge join strategy appropriate for large tables. If the plan still shows nested loop after statistics are current, use a join hint explicitly: `INNER HASH JOIN` or `INNER MERGE JOIN` to override the optimizer decision. Document any queries where join hints are required so they can be revisited when statistics are next updated.

17

STATISTICS AND EXECUTION PLANS

A parameterized stored procedure is fast on first call. Slow on all subsequent calls with different parameters.

THE SITUATION

A stored procedure accepts date range parameters and queries a large Warehouse table. The first execution with a narrow date range completes in 2 seconds. Subsequent calls with wider date ranges take 10 minutes. The same queries run as ad-hoc SQL with the wider ranges complete in 30 seconds.

WRONG THINKING

Engineers test the procedure with various parameters and confirm the slowdown is reproducible. They assume a stored procedure overhead issue and consider rewriting the logic as a view or inline query. The plan cache is not suspected because the first call was fast.

WHAT IS ACTUALLY HAPPENING

This is parameter sniffing. On the first execution, Fabric Warehouse compiles the stored procedure and caches the execution plan optimized for the parameter values used in that first call. A narrow date range produces a plan optimized for low row counts, typically using nested loop joins and index seeks. When the procedure is called with a wide date range that returns millions of rows, it reuses the cached plan built for a small result set. The plan is catastrophically wrong for the actual data volume but is not recompiled because the procedure is already in the plan cache.

THE FIX

Add `OPTION (RECOMPILE)` to the query inside the stored procedure to force recompilation on every execution based on the actual parameter values passed. This eliminates parameter sniffing at the cost of compilation overhead on each call. For procedures called frequently with predictable parameter distributions, consider using `OPTIMIZE FOR UNKNOWN` to build a plan based on average statistics rather than sniffed parameters. Test both options with your actual parameter distribution before deciding which produces the most consistent performance.

18

STATISTICS AND EXECUTION PLANS

Your execution plan looks optimal. Runtime is still 10x slower than expected.

THE SITUATION

A query was analyzed and the execution plan shows efficient operations: hash joins, partition elimination, and reasonable row estimates. By every metric the plan looks correct and efficient. Yet the query takes 10 minutes when based on the plan it should complete in under 1 minute.

WRONG THINKING

Engineers spend time optimizing parts of the query that the plan shows as efficient. Each optimization has little effect. The plan continues to look good while the runtime remains poor. The disconnect between plan quality and actual runtime is confusing.

WHAT IS ACTUALLY HAPPENING

The execution plan shows the logical strategy but not the physical execution bottlenecks. Data skew is invisible in the plan. If one distribution node is processing 80 percent of the data due to a skewed join or group-by key, that node is the bottleneck while all others finish quickly and wait. The plan shows the overall strategy as efficient because the average is reasonable, but the maximum node time dominates total runtime. Skew is one of the most common causes of slow queries that have apparently good execution plans.

THE FIX

Examine per-node execution statistics rather than the overall plan summary. Look for significant variance between the minimum and maximum node execution times. If one node takes 10x longer than the median, data skew is the cause. Identify which column in the join or group-by is skewed by checking value distribution. For skewed join keys, handle dominant key values separately before the main join. For skewed group-by keys, apply salting to distribute the computation across nodes before a final aggregation pass.

19

STATISTICS AND EXECUTION PLANS

Rewriting a query without changing its logic made it 5x faster.

THE SITUATION

Two engineers write logically equivalent queries for the same business requirement. One uses a series of nested subqueries. The other uses CTEs and explicit JOIN conditions. On identical data, the CTE version runs in 1 minute and the nested subquery version runs in 5 minutes. The output of both is identical.

WRONG THINKING

Engineers assume the performance difference is coincidental or caused by plan cache differences between runs. They do not investigate further because both queries return correct results and the faster one is now in use.

WHAT IS ACTUALLY HAPPENING

The Fabric Warehouse query optimizer processes different SQL formulations differently even when they are logically equivalent. Nested correlated subqueries force evaluation order that prevents the optimizer from reordering operations. CTEs with explicit JOINS give the optimizer more freedom to choose join order, distribution strategy, and early filtering. Additionally, deeply nested subqueries can exceed optimizer complexity thresholds causing the planner to fall back to less optimal strategies. The optimizer is not perfectly equivalence-aware and SQL formulation genuinely affects plan quality.

THE FIX

Prefer explicit JOIN syntax over nested subqueries for multi-table queries. Use CTEs to break complex queries into named logical steps rather than nesting subqueries inside subqueries. Avoid correlated subqueries in SELECT lists or WHERE clauses and rewrite them as joins. When two logically equivalent queries produce different performance, compare their execution plans to understand what the optimizer is doing differently. The faster plan reveals which formulation gives the optimizer the most flexibility.

04

FABRIC ENGINEERING PATTERNS

Chapter 4 Concurrency and Blocking

Five scenarios covering how multiple workloads sharing a Fabric Warehouse compete for resources and how blocking, contention, and capacity limits produce unexpected slowdowns.

20

STATISTICS AND EXECUTION PLANS

Two queries returning identical results take completely different amounts of time.

THE SITUATION

Two queries are written by different engineers to answer the same business question. Both return identical row counts and identical values. One takes 30 seconds and the other takes 8 minutes. Neither engineer can explain why their query is slower or faster than the other.

WRONG THINKING

Engineers compare the two queries line by line. The logic looks equivalent. They assume the faster query benefited from a warm plan cache while the slower one did not. After clearing the cache and rerunning both, the performance difference persists.

WHAT IS ACTUALLY HAPPENING

The two queries have identical logical output but different physical execution paths. One query filters early, reducing data before joins. The other filters late, after a full join between large tables. One uses a HASH JOIN hint compatible with the distribution key. The other uses a LOOP JOIN on a non-distributed column. One aggregates before joining reducing row counts dramatically. The other aggregates after joining on the full Cartesian product. Logical equivalence does not imply physical equivalence in a distributed execution engine.

THE FIX

Compare the execution plans of both queries side by side. Identify at which stage each query reduces its row count and how each join is executed. The faster query is reducing data earlier and using more efficient join strategies. Adopt the physical pattern of the faster query even if it requires restructuring the SQL significantly. The key principles are: filter early to reduce rows before joins, aggregate before joining where possible, and ensure join keys align with distribution keys to avoid data movement.

21

CONCURRENCY AND BLOCKING

Your ETL load and Power BI reports run at the same time. Both slow down dramatically.

THE SITUATION

A Fabric Warehouse runs a daily ETL load in the morning that overlaps with business users running Power BI reports. During the overlap window, the ETL takes twice as long as usual and reports time out or return slowly. Outside this window both workloads run at expected speed.

WRONG THINKING

Engineers investigate the ETL pipeline for inefficiencies and the Power BI reports for complex queries. Both look reasonable in isolation. The shared time window is noticed but the connection between the overlap and the slowdown is treated as coincidence rather than cause.

WHAT IS ACTUALLY HAPPENING

The ETL load and Power BI queries are competing for the same Fabric capacity units allocated to the Warehouse. Fabric Warehouse uses a concurrency model where active queries share a pool of compute slots. When the ETL occupies a large portion of those slots with heavy write operations, fewer slots are available for read queries. Power BI queries queue for available slots and experience increased latency. The ETL also experiences contention because its write operations must wait for read locks to release. Both workloads are fighting over the same shared resource pool.

THE FIX

Stagger the ETL load to complete before business users start querying, or shift it to off-peak hours. If overlap is unavoidable, use workload groups or resource classes in the Warehouse to assign dedicated concurrency slots to the ETL versus reporting workloads. Monitor the Fabric Monitoring Hub during the overlap window to confirm which workload is consuming the majority of capacity units and adjust scheduling to reduce peak concurrent demand.

22

CONCURRENCY AND BLOCKING

A long-running query is blocking all other queries in the Warehouse.

THE SITUATION

A Fabric Warehouse query was submitted that takes over an hour to complete. During this run, all other queries submitted by different users queue and do not start. The monitoring view shows pending queries accumulating. As soon as the long query finishes, all pending queries start simultaneously.

WRONG THINKING

Engineers assume a Fabric platform issue is preventing new queries from starting. They attempt to cancel and resubmit queries. Some cancellations succeed but the pending queue continues to grow as long as the long query runs.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse has a finite number of concurrency slots per capacity tier. A single very large query can consume multiple slots depending on its resource requirements. If it consumes enough slots to exhaust the available pool, subsequent queries must wait for a slot to become available. This is not a bug. It is expected resource governance behavior. The long query is not blocking in the traditional row-level lock sense but it is occupying concurrency slots that other queries need to begin execution.

THE FIX

Identify and cancel runaway queries using warehouse monitoring views to find long-running sessions. For recurring long queries, optimize them to run faster or schedule them during periods of low concurrent demand. Consider breaking very large analytical queries into smaller incremental steps that each release concurrency slots between stages. Review whether the Fabric capacity tier provides enough concurrency slots for your peak concurrent workload and upgrade if the slot pool is consistently exhausted during business hours.

23

CONCURRENCY AND BLOCKING

You increased Fabric capacity. Warehouse query performance did not improve proportionally.

THE SITUATION

Fabric capacity was upgraded from one tier to a higher tier expecting a proportional improvement in Warehouse query performance. Individual query times improved slightly but not by the factor expected from the capacity increase. Concurrent query throughput improved more than individual query speed.

WRONG THINKING

Engineers assume the capacity upgrade did not apply correctly to the Warehouse and check workspace settings. The upgrade is confirmed as applied. The expectation that doubling capacity doubles individual query speed is not questioned.

WHAT IS ACTUALLY HAPPENING

Fabric capacity upgrades increase the number of concurrency slots and the total compute available across all workloads in the workspace. For a single query running in isolation, the performance improvement from a capacity upgrade depends on whether that query was previously constrained by compute or by data volume and execution plan quality. A query that is slow because of a poor execution plan, stale statistics, or data skew will not become proportionally faster with more capacity because the bottleneck is not compute availability. Capacity upgrades primarily benefit concurrent workload throughput rather than individual query latency.

THE FIX

Before upgrading capacity to fix a slow query, diagnose whether the bottleneck is compute availability or query design. Check concurrency slot utilization during the slow query. If slots are not fully occupied, adding more capacity will not help. Fix the execution plan, update statistics, address data skew, and optimize the query design first. Reserve capacity upgrades for situations where concurrency slot exhaustion is confirmed as the bottleneck through monitoring data.

24

CONCURRENCY AND BLOCKING

Your morning batch load runs fine alone. When users start querying mid-load, everything slows.

THE SITUATION

A batch load process runs smoothly during off-peak hours. On days when users start querying the Warehouse before the load completes, the load slows significantly and user queries experience high latency. Neither workload causes problems in isolation.

WRONG THINKING

Engineers look for locking conflicts between the load and the queries. When no explicit locks are found, the interaction is attributed to a mysterious platform behavior rather than a resource contention pattern.

WHAT IS ACTUALLY HAPPENING

The batch load consumes a large share of available concurrency slots and I/O bandwidth to write data. User queries arriving mid-load compete for the remaining slots. As more user queries arrive, the slot pool fills and both the load and the queries begin waiting. The load slows because it cannot get consistent access to write slots. User queries slow because read slots are shared with an active heavy write workload. There is no locking in the traditional sense but there is resource contention across the shared capacity pool.

THE FIX

Implement a load status flag in a control table that the batch load sets to active at the start and complete at the end. Configure reporting tools and dashboards to check this flag and display a maintenance message or delay refresh when a load is active. This prevents user queries from competing with the load during the critical window. Alternatively, schedule the load to complete before the business day starts and build enough buffer into the schedule to account for occasional slow runs.

FABRIC ENGINEERING PATTERNS

Chapter 5

SCD and Surrogate Keys

Nine scenarios covering slowly changing dimension patterns, surrogate key management, IDENTITY column behavior, and MERGE performance in Fabric Warehouse engineering environments.

25

CONCURRENCY AND BLOCKING

A single user running a heavy query is consuming all Warehouse concurrency slots.

THE SITUATION

One user submitted a complex analytical query that is consuming the majority of available concurrency slots. All other users in the workspace are unable to run queries and their requests are queuing indefinitely. The heavy query is legitimate but its resource consumption is impacting the entire team.

WRONG THINKING

Workspace administrators look for a way to cancel the query but are unsure of the impact. They consider upgrading capacity as a quick fix. The root cause being a single unoptimized query consuming disproportionate slots is not immediately obvious from the monitoring view.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse allocates concurrency slots based on query resource class or workload group assignment. A user with a large resource class assignment can submit a query that legitimately consumes enough slots to crowd out all other concurrent users. This is by design in the resource governance model. The query is not misbehaving from the platform's perspective. It requested a large resource allocation and received it. The problem is that no governance policy prevents a single user from consuming the full slot pool.

THE FIX

Implement workload groups that cap the maximum concurrency slots any single user or query can consume. Assign users to appropriate resource classes based on their typical query patterns rather than defaulting all users to the maximum resource class. For ad-hoc analytical users, assign smaller resource classes that leave slots available for other concurrent users. Reserve large resource class assignments for scheduled ETL workloads that run during off-peak hours when slot competition is low.

26

SCD AND SURROGATE KEYS

Your SCD Type 1 MERGE is rewriting the entire target table on every run.

THE SITUATION

A daily SCD Type 1 MERGE updates a dimension table with changed attribute values from the source. The MERGE runs correctly but takes far longer than expected given that only a small percentage of rows change each day. The Warehouse monitoring shows massive write volumes on every MERGE execution.

WRONG THINKING

Engineers verify the MERGE logic is correct and confirm only changed rows are being matched and updated. The update logic looks right. The assumption is that MERGE updates only the matched rows and the write volume is unexpectedly high due to a Fabric platform behavior.

WHAT IS ACTUALLY HAPPENING

MERGE in Fabric Warehouse uses copy-on-write at the row group level for columnstore tables. When a MERGE matches rows that span across multiple row groups, each affected row group is fully rewritten even if only one row within it changed. If the dimension table has row groups containing a mix of changed and unchanged rows, the MERGE rewrites every row group that contains at least one match. For large dimension tables with widely distributed changes, nearly all row groups may be rewritten on every run despite only a small fraction of rows actually changing.

THE FIX

For SCD Type 1 dimensions with low daily change rates, consider using UPDATE statements targeting specific rows by primary key rather than a full MERGE. A targeted UPDATE rewrites only the specific rows rather than entire row groups. After updates, run `ALTER INDEX ALL ON tablename REORGANIZE` periodically to consolidate fragmented row groups. For very large dimensions where even targeted updates are slow, evaluate whether a full reload of the changed partition is faster than incremental updates given the copy-on-write overhead.

27

SCD AND SURROGATE KEYS

Your SCD Type 2 implementation is creating duplicate current records.

THE SITUATION

An SCD Type 2 dimension is being maintained by a daily MERGE that inserts new records for changed attributes and closes the previous record by setting an end date and a current flag to false. After several weeks of successful runs, some dimension members have two records with the current flag set to true. Reports depending on the current flag for filtering are returning duplicate rows.

WRONG THINKING

Engineers review the MERGE logic and it looks correct on first inspection. They assume a data quality issue in the source is sending duplicate records and ask the source team to investigate. The MERGE itself is not suspected because it ran without errors.

WHAT IS ACTUALLY HAPPENING

The MERGE has a race condition in its WHEN MATCHED UPDATE clause. The condition to identify the current record for closing uses the current flag and the surrogate key. If the source system sends two changes for the same natural key in the same daily batch, the MERGE processes both in a single execution. The first match closes the current record and inserts a new one. The second match finds the newly inserted record from the same MERGE batch and attempts to update it, but the end-dating logic does not account for records inserted within the same MERGE execution. Both the old and new current records end up with the current flag set to true.

THE FIX

Add a deduplication step before the MERGE that retains only the latest version of each natural key from the source batch. Use a window function like `ROW_NUMBER() OVER (PARTITION BY natural_key ORDER BY change_timestamp DESC)` to identify and keep only the most recent change per key before feeding the source to the MERGE. Add a post-MERGE data quality check that counts records where current flag is true and the count per natural key is greater than 1, and fail the pipeline if any duplicates are detected.

28

SCD AND SURROGATE KEYS

SCD Type 2 end-dating logic is missing rows. Some records have no end date and no current flag.

THE SITUATION

An SCD Type 2 dimension table has records that are neither closed with an end date nor marked as current. These orphaned records have a null end date and a false current flag. They are invisible to reports filtering for current records and invisible to history queries filtering for records within a date range. They exist in the table but are effectively unreachable.

WRONG THINKING

Engineers assume the orphaned records are a data quality issue from the source and look for irregularities in the source change data. When the source looks clean, the MERGE logic is reviewed but appears correct for the scenarios it was designed to handle.

WHAT IS ACTUALLY HAPPENING

The MERGE was designed to handle two cases: new natural keys not in the dimension, and existing natural keys with changed attributes. A third case was not handled: records that exist in the dimension with a current flag of true but are no longer present in the source system at all. When a source record is deleted or deactivated, the MERGE does not match it in the WHEN MATCHED clause because the source row is absent. The current record in the dimension is never closed and never updated. It becomes an orphan with no end date and current flag eventually set to false by a partial update that did not complete the end-dating.

THE FIX

Add a WHEN NOT MATCHED BY SOURCE clause to the MERGE that handles source deletions explicitly. For SCD Type 2, this clause should close the current dimension record by setting the end date to the current date and setting the current flag to false. Test this clause with a representative sample of source deletions before deploying to production. Add a scheduled audit query that counts dimension records with null end date and false current flag and alerts when the count exceeds zero.

29

SCD AND SURROGATE KEYS

Your SCD Type 2 history table is growing 10x faster than expected. Storage is ballooning.

THE SITUATION

An SCD Type 2 dimension was designed to track attribute changes over time. After 6 months in production, the table has 50 times more rows than the number of dimension members. Storage costs are significantly higher than projected. A sample of the data reveals many dimension members have hundreds of historical versions.

WRONG THINKING

Engineers assume the source system has unusually high change frequency and accept the storage growth as a data characteristic rather than a design problem. The number of versions per dimension member is not analyzed in detail.

WHAT IS ACTUALLY HAPPENING

The source system is sending full snapshots rather than change deltas. Every daily load includes all dimension members regardless of whether their attributes changed. The MERGE compares each source row against the current dimension record and because the comparison logic has a subtle flaw, it treats unchanged records as changed and inserts new SCD Type 2 versions for every member on every run. After 180 daily runs, each dimension member has up to 180 historical versions when the attributes may not have actually changed at all.

THE FIX

Add a change detection step before the MERGE that compares source attribute values against current dimension values and filters out records where no attributes changed. Use a hash comparison of all tracked attribute columns to efficiently identify unchanged records. Only records where the hash differs should enter the MERGE as candidates for SCD Type 2 inserts. After fixing the change detection logic, purge the excess historical versions created during the period when the logic was incorrect, retaining only legitimate changes.

30

SCD AND SURROGATE KEYS

Your IDENTITY column is generating duplicate key values after a parallel load.

THE SITUATION

A Fabric Warehouse table uses an IDENTITY column as the surrogate key. A parallel load process inserts rows into the table from multiple concurrent sessions. After the load, primary key violation errors appear in downstream processes that join on the surrogate key. Checking the table reveals duplicate IDENTITY values.

WRONG THINKING

Engineers assume the IDENTITY implementation in Fabric Warehouse has a bug and raise a support request. The expectation is that IDENTITY columns guarantee uniqueness by definition, making the duplicates feel like a platform defect.

WHAT IS ACTUALLY HAPPENING

IDENTITY columns in Fabric Warehouse generate values based on range allocation per session. Each parallel session receives a range of IDENTITY values to use for its inserts. If the load process inserts rows from multiple sessions that each received overlapping ranges due to incorrect session management, duplicate values can be generated. Additionally, if a load session fails mid-execution and is retried, the IDENTITY ranges allocated to the failed session may be reused if the session state was not properly cleaned up before retry. IDENTITY does not guarantee uniqueness across parallel sessions without proper session isolation.

THE FIX

For parallel loads, use a single session for IDENTITY-keyed inserts or coordinate IDENTITY range allocation explicitly across sessions. Consider replacing IDENTITY with a hash-based surrogate key generated from the natural key columns using `HASHBYTES` or a similar deterministic function. Hash-based surrogates are parallel-safe because the same natural key always produces the same surrogate value regardless of which session generates it. Add a post-load uniqueness check on the surrogate key column before committing the load to downstream consumers.

31

SCD AND SURROGATE KEYS

After truncating and reloading a table, IDENTITY values restart. Existing foreign key references break.

THE SITUATION

A large dimension table was truncated and fully reloaded from source as part of a periodic refresh. After the reload, IDENTITY surrogate key values restarted from 1. Fact tables that had stored the old surrogate key values as foreign keys now reference keys that either do not exist or point to completely different dimension members.

WRONG THINKING

Engineers assume truncating and reloading a table is a safe full refresh operation. The IDENTITY restart is not anticipated because SQL Server IDENTITY columns do not automatically reseed after truncate in all configurations. The foreign key breakage is discovered hours later when downstream reports return wrong results.

WHAT IS ACTUALLY HAPPENING

In Fabric Warehouse, truncating a table resets the IDENTITY seed to the original starting value. Unlike SQL Server where `TRUNCATE TABLE` resets the IDENTITY counter, some engineers expect the counter to continue from its previous maximum. When fact tables store surrogate keys as foreign keys and the dimension is truncated and reloaded, the new IDENTITY values starting from 1 collide with or replace the old surrogate keys that the fact tables reference, producing incorrect joins or missing matches.

THE FIX

Never truncate and reload a dimension table that has surrogate keys referenced by fact tables without first updating or rebuilding the fact tables to use the new surrogate keys. For full dimension refreshes, use a staging table pattern: load new data into a staging table, generate new surrogate keys there, update fact tables to reference the new keys, then swap the staging table into production. Alternatively, use hash-based surrogate keys derived from natural keys so that the surrogate key value is deterministic and does not change between reloads regardless of IDENTITY behavior.

32

SCD AND SURROGATE KEYS

You switched from IDENTITY to hash-based surrogate keys. Hash collisions are appearing.

THE SITUATION

A decision was made to replace IDENTITY surrogate keys with hash-based keys generated from the natural key columns using a hashing function. After deployment, a small number of fact table rows have incorrect dimension joins. Investigation reveals two different natural keys are producing the same hash value.

WRONG THINKING

Engineers assume hash collisions are mathematically impossible or vanishingly rare for business data. The collision rate feels like a platform bug rather than an expected property of hash functions applied to real data.

WHAT IS ACTUALLY HAPPENING

Hash functions can produce collisions when applied to string-based natural keys that differ in ways the hash function does not distinguish. Using a 32-bit hash like `CHECKSUM` on a large number of dimension members produces collisions at a meaningful rate because 32 bits provide only about 4 billion distinct values. A dimension with tens of millions of members has a statistically meaningful chance of collision. Using `HASHBYTES` with MD5 or SHA produces a much larger hash space but the output is a binary value that requires additional handling to use as an integer surrogate key.

THE FIX

Use `HASHBYTES('SHA2_256', natural_key_columns)` rather than `CHECKSUM` for surrogate key generation. Convert the binary output to a bigint using a consistent algorithm. Add a uniqueness constraint or a post-load check that validates no two distinct natural keys produced the same hash value before the load is committed. For dimensions where natural keys are composite, concatenate all key columns with a separator character before hashing to ensure distinct combinations produce distinct hashes.

33

SCD AND SURROGATE KEYS

Your MERGE statement for SCD Type 2 is inserting new rows but not updating the `is_current` flag on old rows.

THE SITUATION

A MERGE-based SCD Type 2 process inserts new version records correctly for changed dimension members. But the previous version records are not being closed. The `is_current` flag on old records remains true and the `end_date` remains null. Every changed member now has two current records.

WRONG THINKING

Engineers review the MERGE statement and see both WHEN MATCHED and WHEN NOT MATCHED clauses. The WHEN NOT MATCHED clause inserts new rows. The WHEN MATCHED clause appears to update old rows. The logic looks correct at first reading.

WHAT IS ACTUALLY HAPPENING

The MERGE WHEN MATCHED clause is matching on the surrogate key rather than the natural key. The new surrogate key for the inserted version does not exist in the target yet, so the WHEN MATCHED condition never fires for the old version. The old version is simply not matched by the MERGE because the MERGE join is on the wrong column. The WHEN NOT MATCHED clause inserts the new version correctly but the update that should close the old version never executes because nothing matches the old surrogate key in the source.

THE FIX

For SCD Type 2 MERGE statements, the WHEN MATCHED condition must join on the natural key to find the existing current version, not on the surrogate key. The WHEN MATCHED UPDATE clause closes the old version by setting `end_date` and `is_current`. The WHEN NOT MATCHED INSERT clause creates the new version with a new surrogate key. These are two separate operations on the same natural key. Always test the MERGE with a small controlled dataset where you can verify both the insert of the new version and the update of the old version before running against data at scale.

34

SCD AND SURROGATE KEYS

Your MERGE runs correctly on 1 million rows. It falls over on 100 million.

THE SITUATION

A MERGE statement that maintains a large fact or dimension table runs correctly in testing on a 1 million row subset. When run against the full 100 million row production table, it either times out, runs for hours, or fails with a resource error. The logic has not changed between test and production.

WRONG THINKING

Engineers assume the engineering environment needs more capacity and request a warehouse upgrade. When the upgrade does not resolve the issue, the MERGE logic itself is examined but appears correct. The scale difference between test and production is noted but the specific reason for the non-linear performance degradation is not identified.

WHAT IS ACTUALLY HAPPENING

MERGE on large tables in Fabric Warehouse performs a full scan of both the source and target tables to identify matches, updates, and inserts. At 1 million rows the full scan is fast. At 100 million rows the full scan is expensive and the data movement required to redistribute both datasets across distribution nodes for matching multiplies the cost. Additionally, the MERGE must hold row-level locks or row group locks during execution which increases contention. The operation does not scale linearly because the join cost grows with both table sizes simultaneously.

THE FIX

Partition the MERGE operation to process one logical partition at a time rather than the entire table in a single statement. Use a wrapper that iterates over date partitions or key ranges and executes a targeted MERGE for each partition. This reduces the data volume per MERGE execution, lowers lock contention, and allows failed partitions to be retried without reprocessing the entire table. Add a WHERE clause to both the source and target sides of the MERGE to explicitly limit the scan to the current partition range.

FABRIC ENGINEERING PATTERNS

Chapter 6

SQL Patterns

Ten scenarios covering views, stored procedures, window functions, subqueries, and CTEs in Fabric Warehouse and where T-SQL constructs behave differently from SQL Server expectations.

35

SQL PATTERNS

You tried to create a temp table inside a stored procedure. It does not behave like SQL Server.

THE SITUATION

A stored procedure was written for Fabric Warehouse using the same T-SQL pattern used in SQL Server, including temp table creation with the # prefix for intermediate result storage. The procedure fails to compile or produces unexpected errors when executed. The same code works correctly in SQL Server.

WRONG THINKING

Engineers assume a syntax error in their code and try variations of the temp table creation statement. When the error persists regardless of syntax, the assumption shifts to a Fabric platform bug. The possibility that temp table behavior differs between SQL Server and Fabric Warehouse is not immediately considered.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse has limited support for session-scoped temp tables compared to SQL Server. The T-SQL temp table syntax using # prefix may not be fully supported or may behave differently in the context of stored procedures in Fabric Warehouse. Fabric Warehouse is not a drop-in replacement for SQL Server and does not implement the full T-SQL surface area. Certain procedural T-SQL features including temp tables, table variables in some contexts, and dynamic SQL have different behavior or limited support.

THE FIX

Replace temp tables with Common Table Expressions where the logic allows, or use permanent staging tables in the warehouse schema with a naming convention that identifies them as transient. Create the staging table at the start of the procedure, populate it, use it, and truncate or drop it at the end. For complex multi-step procedures, break the logic into multiple separate queries or use a Fabric notebook for procedural logic that requires iterative temp storage, reserving stored procedures for set-based operations that do not require intermediate session state.

36

SQL PATTERNS

Your stored procedure has a transaction block. It behaves differently than expected in Fabric Warehouse.

THE SITUATION

A stored procedure wraps multiple INSERT and UPDATE statements in an explicit transaction using BEGIN TRANSACTION and COMMIT TRANSACTION. In SQL Server this guarantees all operations succeed or all are rolled back. In Fabric Warehouse, the procedure completes without error but partial updates are visible during execution and rollback behavior does not match SQL Server expectations.

WRONG THINKING

Engineers assume explicit transaction blocks in stored procedures provide identical ACID guarantees in Fabric Warehouse as in SQL Server. When partial updates become visible mid-execution or rollback does not restore all affected rows, the assumption is a Fabric platform defect.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse is a distributed MPP system with ACID guarantees at the statement level. Multi-statement transaction support in stored procedures has different behavior compared to SQL Server's single-node transaction model. BEGIN TRANSACTION and COMMIT are supported but the isolation guarantees and rollback behavior across distributed nodes may differ from single-node SQL Server behavior. Long-running distributed transactions also carry higher risk of partial failure than equivalent SQL Server transactions.

THE FIX

Design stored procedures to be idempotent rather than relying on transaction rollback for correctness. Write procedures so that if they are interrupted and rerun from the beginning, they produce the same correct result. Use staging tables and swap patterns rather than in-place multi-statement updates. For operations that genuinely require atomicity, test rollback behavior explicitly in Fabric Warehouse with your specific operations before relying on it in production. Document any differences from SQL Server behavior found during testing.

37

SQL PATTERNS

A view that works correctly returns wrong results when filtered in a query.

THE SITUATION

A view is created over a large fact table with several joins and calculated columns. When the view is queried without filters it returns correct results. When a WHERE clause filter is applied to the query against the view, the results are wrong. Some rows that should be included are missing and some that should be excluded appear.

WRONG THINKING

Engineers check the view definition and confirm it looks correct. They apply the same filter directly to the underlying table query and get the correct result. The discrepancy between filtering the view and filtering the base query is unexpected.

WHAT IS ACTUALLY HAPPENING

The view contains a filter condition in its own WHERE clause that interacts with the outer filter in an unexpected way. A view with a WHERE clause that filters on a column that is also used in the outer query filter can produce unexpected results when the filter conditions use different expressions for the same logical concept, such as a date truncated to month in the view and a full date comparison in the outer query. Additionally, if the view uses outer joins, applying a filter on a nullable column from the outer join in the outer WHERE clause effectively converts the outer join to an inner join, excluding rows that the view intended to include.

THE FIX

When a view produces unexpected results under filtering, expand the view definition inline into the query and apply the filter there to see if results match. If they differ, the view definition is the problem. Audit the view for WHERE clauses that interact with outer filters, outer joins that are affected by outer WHERE conditions, and calculated columns that change behavior under certain filter conditions. Rewrite the view to be filter-safe by ensuring outer join semantics are preserved and filter conditions are clearly scoped.

38

SQL PATTERNS

Your view references a table that was renamed. It fails silently and returns no error.

THE SITUATION

A table in the Fabric Warehouse was renamed as part of a refactoring effort. Several views reference this table. After the rename, the views continue to return results without throwing errors. But the results are wrong. Some views are returning stale data and others are returning data from an unexpected source.

WRONG THINKING

Engineers assume the views automatically updated their references when the table was renamed. Since no errors appear, the view is assumed to be working correctly. The wrong results are attributed to data quality issues rather than a broken view reference.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse view definitions are stored as text at creation time. When the underlying table is renamed, the view definition is not automatically updated. Depending on how the rename was handled, the view may resolve to a different object that happens to have the old name, or it may return results from a cached plan that has not been invalidated. The absence of an error does not confirm the view is working correctly. Silent resolution to an unintended object is a known risk when renaming objects that views depend on.

THE FIX

Before renaming any table, identify all views that reference it using system catalog views that query object dependencies. Update all dependent views to reference the new table name before or immediately after the rename. After updating, test each affected view to confirm it returns expected results. Establish a convention that table renames require a view dependency audit as a mandatory step in the rename process. Never assume view definitions self-update when underlying objects are renamed.

39

SQL PATTERNS

A window function works in a standalone query. It fails when placed inside a view.

THE SITUATION

A window function using `ROW_NUMBER` with `PARTITION BY` and `ORDER BY` returns correct results when run as a standalone `SELECT` statement. When the same window function expression is embedded inside a view definition, the view either fails to create, throws an error on query, or returns results that differ from the standalone version.

WRONG THINKING

Engineers copy the exact window function expression from the working standalone query into the view definition. The expression is identical. The failure inside the view is attributed to a Fabric platform inconsistency rather than a structural difference between how window functions behave in views versus standalone queries.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse has restrictions on certain T-SQL constructs inside view definitions. Window functions with `ORDER BY` clauses inside views may require the view to have a corresponding `TOP` or `FETCH` clause in some contexts, similar to SQL Server restrictions. Additionally, if the view is used inside another query that applies further filtering or aggregation, the window function partitioning may produce different results depending on the row set the outer query presents to the view versus the full table the standalone query operates on. The window function evaluates over whatever rows the view sees at query time, not over the full table.

THE FIX

Test window functions inside views with representative outer queries that include filters and aggregations matching production usage. If the view fails to create with a window function, add a wrapper CTE inside the view definition that isolates the window function in its own named scope before the outer select. For views that are frequently filtered by outer queries, validate that the window function partitioning produces correct results when the outer filter reduces the row set significantly.

40

SQL PATTERNS

Your window function with PARTITION BY is correct in dev. Returns different results on full data at scale.

THE SITUATION

A window function using RANK or DENSE_RANK with PARTITION BY a category column and ORDER BY a value column returns correct rankings in development testing. In production with the full dataset, the rankings for some categories are wrong. Certain records receive the same rank as others they should outrank.

WRONG THINKING

Engineers run the same query in dev with the exact same parameters and confirm it returns correct rankings. They assume a data quality issue in production and compare source data between environments. The data distributions look similar. The query is not suspected.

WHAT IS ACTUALLY HAPPENING

The ORDER BY clause in the window function references a column that has duplicate values within some partition groups in data at scale. When multiple rows have identical ORDER BY values within a partition, the ranking function assigns the same rank to all tied rows. In dev with a small curated dataset, ties may not exist or may be rare. In full data at scale with high-volume categories, ties are common. The ranking is technically correct according to the function definition but the business expectation assumed uniqueness within each partition that does not hold in production.

THE FIX

Add a tiebreaker column to the ORDER BY clause of the window function to ensure unique ordering within each partition. A surrogate key, a timestamp, or any column that is unique within the partition will prevent ties. If ties should be handled with a specific business rule such as first-in wins, order by a timestamp column. If true ranking with tied positions is the requirement, use DENSE_RANK and document that tied positions are expected behavior. Validate the ranking logic with a data at scale sample that includes high-frequency category values before deploying.

41

SQL PATTERNS

A correlated subquery that works on small data hangs on data at scale with no error.

THE SITUATION

A query uses a correlated subquery in the SELECT list to fetch a single aggregated value for each row in the outer table. On dev data with 10,000 rows the query completes in seconds. On production with 5 million rows it runs for hours without completing or producing an error. The session eventually times out.

WRONG THINKING

Engineers test the subquery independently and confirm it returns results quickly. They assume the subquery is efficient because it ran fast in isolation and that production slowness must be caused by something else in the query.

WHAT IS ACTUALLY HAPPENING

A correlated subquery in the SELECT list executes once for every row returned by the outer query. With 5 million outer rows, the subquery executes 5 million times. Each execution may require a full or partial scan of the subquery source table. Even if each individual execution takes 10 milliseconds, 5 million executions total 50,000 seconds of compute time. The subquery is fast in isolation because it runs once. In the correlated context it runs millions of times and the cost compounds with every row in the outer result.

THE FIX

Rewrite the correlated subquery as a JOIN to a pre-aggregated subquery or CTE. Compute the aggregation once across all groups in a CTE, then join the CTE to the outer table on the grouping key. This transforms millions of individual subquery executions into a single aggregation pass followed by one join. The result is identical but the execution cost is orders of magnitude lower. This rewrite pattern applies to any correlated subquery that computes an aggregate per outer row.

42

SQL PATTERNS

Your stored procedure compiles without errors. It returns wrong results when parameters contain NULL values.

THE SITUATION

A stored procedure accepts parameters for filtering and returns aggregated results. It compiles and executes without errors. When called with NULL parameter values representing unfiltered requests, the result set is empty or returns fewer rows than expected. The same query with NULL replaced by a specific value returns correct results.

WRONG THINKING

Engineers test the procedure with various non-NULL parameters and confirm it works correctly. NULL parameter handling is not tested explicitly because the procedure compiled cleanly and the business requirement for NULL as a wildcard is understood informally rather than documented in the code.

WHAT IS ACTUALLY HAPPENING

The WHERE clause in the stored procedure uses direct equality comparison on the parameter. A condition like `WHERE column = @parameter` evaluates to UNKNOWN rather than TRUE when @parameter is NULL because NULL equals nothing in SQL, including NULL itself. UNKNOWN conditions filter out all rows. The procedure was written assuming parameters would always have non-NULL values but was called with NULL intending a wildcard match that returns all rows. The logic gap between NULL as no-filter and NULL as unknown is a classic T-SQL trap.

THE FIX

Rewrite the WHERE clause to handle NULL parameters explicitly using an OR IS NULL pattern: `WHERE (@parameter IS NULL OR column = @parameter)`. This returns all rows when the parameter is NULL and filters normally when it has a value. Alternatively, use `COALESCE(@parameter, column)` to substitute the column value for NULL parameters, which always evaluates to true for equality. Add NULL parameter test cases to the procedure testing checklist and document the expected behavior for NULL inputs explicitly in the procedure header comments.

43

SQL PATTERNS

T-SQL syntax that works in SQL Server fails in Fabric Warehouse with a cryptic error.

THE SITUATION

A T-SQL script migrated from SQL Server to Fabric Warehouse fails with an error that does not clearly identify which part of the syntax is unsupported. The script uses standard T-SQL constructs that have been in SQL Server for many versions. The error message refers to a parser or syntax issue without specifying the unsupported feature.

WRONG THINKING

Engineers search for the exact error message and find no clear documentation. They assume a Fabric platform bug or a version incompatibility and try minor syntax variations hoping one will work. The unsupported T-SQL feature is not identified because the error does not name it explicitly.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse implements a subset of T-SQL, not the full SQL Server T-SQL surface area. Features not supported include certain DDL statements, some procedural T-SQL constructs, specific system functions, certain cursor operations, and some advanced query hints. The T-SQL parser in Fabric Warehouse may return a generic syntax error rather than a specific unsupported feature message, making it difficult to identify exactly which construct is causing the failure without methodically isolating each statement.

THE FIX

When a migrated script fails, isolate the failure by executing statements individually rather than the full script at once. The statement that produces the error contains the unsupported construct. Cross-reference the failing statement against the Fabric Warehouse T-SQL coverage documentation which lists supported and unsupported features. For unsupported procedural constructs, rewrite the logic using set-based SQL patterns or move the procedural logic to a Fabric notebook. Keep a migration checklist of constructs validated as working in Fabric Warehouse for your team to reference.

44

SQL PATTERNS

A recursive CTE that works in SQL Server hits a recursion limit error in Fabric Warehouse.

THE SITUATION

A recursive CTE used for hierarchical data traversal, such as an org chart or a bill of materials, works correctly in SQL Server. When the same query is run in Fabric Warehouse, it either hits a recursion limit error or returns incomplete results that stop at a shallower hierarchy depth than the actual data.

WRONG THINKING

Engineers assume the recursion limit in Fabric Warehouse is set lower than SQL Server and look for a MAXRECURSION query hint to increase it. When the hint does not resolve the issue or is not supported, the assumption shifts to a Fabric platform limitation with no workaround.

WHAT IS ACTUALLY HAPPENING

Fabric Warehouse supports recursive CTEs but the default MAXRECURSION limit and the behavior under deep recursion may differ from SQL Server. For hierarchies with more than 100 levels, the default recursion limit is exceeded. Additionally, recursive CTEs in a distributed MPP environment like Fabric Warehouse have different performance characteristics than in SQL Server because each recursion level may require data distribution operations across nodes, making deep recursion expensive and potentially unstable.

THE FIX

Add `OPTION (MAXRECURSION 0)` to remove the recursion limit if the query hint is supported in Fabric Warehouse for your use case. For very deep hierarchies, consider flattening the hierarchy into a separate table using an iterative process in a Fabric notebook rather than relying on recursive SQL. Store the pre-computed parent-child paths in a hierarchy table that is refreshed when the source hierarchy changes. This trades real-time recursive traversal for a pre-computed lookup that performs consistently regardless of hierarchy depth.

FABRIC ENGINEERING PATTERNS

Chapter 7

Ingestion and Cross-Warehouse

Six scenarios covering data loading patterns, cross-warehouse queries, SQL endpoint behavior, and the specific ways Fabric Warehouse differs from SQL Server when data moves across boundaries.

45

INGESTION AND CROSS-WAREHOUSE

Data written to your Lakehouse is not visible in the SQL endpoint immediately.

THE SITUATION

A pipeline writes data to a Lakehouse Delta table and then queries the same table through the Fabric SQL analytics endpoint in a subsequent step. The query returns zero rows or data from a previous load. When the same table is queried directly through the Lakehouse notebook interface, the new data is visible immediately.

WRONG THINKING

Engineers assume the SQL endpoint and the Lakehouse notebook are reading from the same data and should return identical results at any point in time. The lag between write and SQL endpoint visibility is not expected and is attributed to a Fabric bug.

WHAT IS ACTUALLY HAPPENING

The Fabric SQL analytics endpoint for a Lakehouse does not read Delta files directly in real time. It uses a metadata synchronization process that periodically updates the SQL endpoint's schema and statistics to reflect the current state of the Delta tables in OneLake. The synchronization interval means there is a lag between when data is written to a Delta table and when that data is queryable through the SQL endpoint. The notebook reads Delta files directly and always sees the latest committed data. The SQL endpoint reads through the synchronized metadata layer which may be behind by minutes.

THE FIX

Do not rely on the SQL endpoint for immediate post-write validation in pipeline activities. Use a notebook activity or a Spark SQL query that reads the Delta table directly for post-load row count checks. If downstream SQL endpoint consumers require near-real-time data, check the current sync lag for your workspace and design the pipeline to wait for synchronization to complete before triggering downstream SQL endpoint queries. For critical pipelines, add an explicit sync trigger if available or add a timed wait activity that provides sufficient buffer for the synchronization to propagate.

46

INGESTION AND CROSS-WAREHOUSE

A query joining a Warehouse table and a Lakehouse table via shortcut takes 20x longer than either alone.

THE SITUATION

A Fabric Warehouse contains a shortcut to a Lakehouse Delta table. A query joins the Warehouse native table with the shortcut Lakehouse table. Both tables individually return results quickly. The join between them takes orders of magnitude longer than either table alone.

WRONG THINKING

Engineers optimize the query by adding filters and reducing selected columns. Some improvement is seen but the cross-source join remains dramatically slower than expected. The join condition is verified as correct and the tables are confirmed to be of similar size.

WHAT IS ACTUALLY HAPPENING

Joining a Warehouse native table with a Lakehouse table accessed through a shortcut requires data movement between two different execution engines. The Warehouse native table is read by the Warehouse MPP engine. The Lakehouse shortcut table is read through the Delta engine via OneLake. Combining results from two different engines requires one side to be materialized and transferred to the other engine for the join. This cross-engine data movement is significantly more expensive than a join between two tables in the same engine. The individual table reads are fast because each engine reads its own data efficiently. The join is slow because data must cross the engine boundary.

THE FIX

Materialize the Lakehouse data into a native Warehouse table by copying it with a COPY INTO or CTAS statement rather than accessing it through a shortcut at query time. A native Warehouse table joins with other native tables using the MPP engine's full distributed join capabilities without cross-engine data movement. Schedule the materialization to refresh after each Lakehouse load so the Warehouse copy stays current. Accept the trade-off of a slightly stale copy in exchange for dramatically better join performance.

47

INGESTION AND CROSS-WAREHOUSE

Your cross-warehouse query works in dev. Fails with permission errors in production.

THE SITUATION

A query that references tables in a different Fabric Warehouse using cross-warehouse syntax works correctly in the development workspace. When the same query is deployed to production, it fails with permission or access errors even though the production Warehouse contains identical data and schema.

WRONG THINKING

Engineers check that the production Warehouse tables exist and have the same names. They confirm the query syntax is identical between dev and prod. The permission error is attributed to a Fabric workspace configuration issue that needs IT intervention.

WHAT IS ACTUALLY HAPPENING

Cross-warehouse queries in Fabric require the querying identity to have explicit access permissions to the target Warehouse in addition to the source. In development, the engineer running the query has personal permissions granted to both workspaces. In production, the query runs under a service principal or scheduled identity that has permissions in the source workspace but not in the target warehouse workspace. The query works in dev because the developer has implicit access. It fails in prod because the service identity was never granted access to the cross-workspace target.

THE FIX

Grant the production service principal or managed identity the appropriate read permissions on the target Warehouse in the target workspace. The minimum required permission is typically a read role on the specific tables being queried. After granting permissions, test the cross-warehouse query using the same identity that the production pipeline uses rather than using personal developer credentials. Document all cross-workspace permission dependencies as part of the production deployment checklist.

48

INGESTION AND CROSS-WAREHOUSE

Loading data row by row into Fabric Warehouse takes hours. Bulk load takes seconds.

THE SITUATION

A data loading process inserts rows into a Fabric Warehouse table using individual INSERT statements in a loop, one row per statement. The load runs overnight and takes 6 to 8 hours for 2 million rows. When the same data is loaded using a single bulk INSERT or COPY INTO statement, it completes in under a minute.

WRONG THINKING

Engineers assume the row-by-row approach is acceptable because it works correctly and produces the right data. Performance is considered a secondary concern until the load time grows to the point of missing the business deadline.

WHAT IS ACTUALLY HAPPENING

Each individual INSERT statement in Fabric Warehouse is a separate transaction that involves statement parsing, plan compilation, distributed execution, transaction commit, and log write. For a single row, these fixed costs dominate the actual data transfer cost. Multiplied across 2 million rows, the overhead is enormous. Fabric Warehouse, like all distributed SQL systems, is optimized for set-based bulk operations not row-by-row procedural loops. The per-statement overhead that is negligible for large batch operations becomes the dominant cost when applied to individual rows.

THE FIX

Replace row-by-row INSERT loops with set-based bulk load patterns. Use `INSERT INTO ... SELECT` with the full dataset, `COPY INTO` from staged files in OneLake, or a pipeline copy activity that writes in bulk. For scenarios where rows arrive individually from a streaming source, batch them into groups of at least 10,000 rows before inserting. The performance difference between row-by-row and bulk loading in Fabric Warehouse is not marginal. It is typically 100x to 1000x in favor of bulk operations.

49

INGESTION AND CROSS-WAREHOUSE

You truncated and reloaded a Warehouse table. Queries are now slower than before the reload.

THE SITUATION

A Fabric Warehouse table was truncated and fully reloaded from source as part of a weekly refresh. The data is identical to before the reload. Row counts match. But queries that ran in 30 seconds before the reload now take several minutes. The slowdown persists across multiple query executions after the reload.

WRONG THINKING

Engineers check that all rows were loaded correctly and confirm the data matches the source. They assume a temporary issue and wait for performance to self-correct. When it does not improve over several hours, the reload process itself is not suspected as the cause.

WHAT IS ACTUALLY HAPPENING

Truncating and reloading a Fabric Warehouse table clears the table statistics that had been built up over time. After the reload, the table has fresh data but no statistics, or statistics that were rebuilt only partially during the load. The query planner has no accurate information about row counts, column distributions, or data skew. It falls back to default estimates that may be dramatically wrong for the actual data, producing suboptimal execution plans. The table before the reload had mature statistics built from many update cycles. The reloaded table starts from scratch.

THE FIX

After every full table reload, immediately run `UPDATE STATISTICS tablename` with full scan before any production queries execute against the table. Add this as a mandatory step in the reload pipeline between the data load completion and the downstream notification that the table is ready. For large tables where full statistics updates are slow, run statistics on the most critical filter and join columns first, then schedule full statistics updates to run during the next maintenance window.

50

INGESTION AND CROSS-WAREHOUSE

Your COPY INTO loaded data correctly but aggregations on the loaded data are wrong.

THE SITUATION

A COPY INTO statement successfully loads a large file from OneLake into a Fabric Warehouse table. Row counts match the source file. But aggregate queries on the loaded data, such as SUM and COUNT DISTINCT, return values that differ from equivalent aggregations run directly on the source file in a notebook.

WRONG THINKING

Engineers verify the row count match and conclude the data loaded correctly. The aggregation discrepancy is attributed to a data quality issue in the source file. When the source file is checked directly, the aggregations match the expected values. The COPY INTO process is not suspected because it reported success with matching row counts.

WHAT IS ACTUALLY HAPPENING

COPY INTO may perform implicit type casting on numeric and decimal columns when the inferred or specified column types do not exactly match the source data precision. A decimal column stored as DECIMAL(18,6) in the source file loaded into a DECIMAL(10,2) Warehouse column will be silently truncated to 2 decimal places. The row count is unaffected but the numeric precision loss produces different aggregation results. The COPY INTO reported success because truncation is handled as a type coercion rather than an error.

THE FIX

Define explicit column type mappings in the COPY INTO statement that match the source data precision exactly. For numeric and decimal columns, verify the source file precision before defining the target column type in the Warehouse. After loading, add a post-load validation that compares key aggregate values between the source file queried through a notebook and the loaded Warehouse table. Any discrepancy in aggregations after a successful row count match indicates a precision or type casting issue in the load configuration.

CONCLUSION

The SQL you know is not the SQL here.

Across 50 scenarios, one pattern repeats. The engineers who struggle most with Fabric Warehouse are the ones with the most SQL Server experience. Not because their knowledge is wrong, but because it is precise enough to be misleading. Indexes that do not exist. Statistics that behave differently. Transactions that do not behave like OLTP transactions. Fabric Warehouse accepts familiar syntax but runs on a different engine, and that changes almost everything about how queries perform and how data should be modeled.

ALSO IN THE SERIES

Pipelines and Data Factory

- Your pipeline succeeded. The target table is empty.
- Your trigger was active. It silently stopped firing months ago.

Power BI in Fabric

- Your Direct Lake report fell back to DirectQuery silently after a schema change.
- RLS is defined correctly. Users are still seeing data they should not.

And more. Free at ssanjaychandra.com

A NOTE ON THIS RELEASE

This is the initial release of the Fabric Engineering Patterns series. The patterns and fixes reflect the platform as it stands today. Microsoft Fabric evolves quickly and some guidance may need updating as the platform matures. If you spot a technical inaccuracy, a pattern that has been superseded by a platform update, or a scenario that deserves a deeper treatment, your feedback is genuinely welcome. Reach out through any of the channels below.

CONNECT

-  www.ssanjaychandra.com
-  linkedin.com/in/ssanjaychandra
-  ssanjaychandra@outlook.com

Download all books in this series free at ssanjaychandra.com/freedownloads

Sanjay Chandra

The Databricks + Fabric guy on LinkedIn